

count 정리

범위 안에서 특정 값이 몇 번 나오는지 세는 알고리즘입니다.

사용처 (Where)

- 특정 값의 등장 횟수 계산
- 중복 여부 확인
- 간단한 빈도 확인

방법 (How to Use)

- begin/end 범위와 값을 넘김
- 반환값은 등장 횟수
- 조건 기반은 count_if 사용
- 큰 빈도표는 map/unordered_map 고려

기본 정보

- 필요 헤더: #include <algorithm>
- 기본 선언: int n = count(v.begin(), v.end(), x);

왜 써야 할까? (Why)

- 짧은 코드로 개수를 바로 구함
- 조건식이 필요하면 count_if로 확장 가능
- 반복문보다 목적이 명확함

주요 함수

함수/문법	의미
count(first,last,x)	x 개수
count_if	조건 만족 개수
반환값	정수형 개수

기본 코드 예시

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1, 2, 2, 3};
    cout << count(v.begin(), v.end(), 2);
}
```

count 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
count	$O(n)$
count_if	$O(n)$
map 기반 빈도	삽입당 $O(\log n)$

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	count 함수 호출만 알면 됨	반복문과 조건을 직접 설계	qsort/직접 반복문 사용
코드 길이	짧고 의도가 분명함	길어지고 실수 지점 증가	자료형 처리와 캐스팅이 번거로움
안정성	표준 라이브러리라 검증됨	구현 실수 가능	포인터/범위 실수 가능
추천 상황	대부분의 일반 상황	원리를 공부하거나 특수 최적화	C 코드와 연동할 때

응용: 짝수 개수 세기

count_if와 lambda를 사용하면 특정 조건을 만족하는 원소를 셀 수 있습니다.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4, 5, 6};
    int even = count_if(v.begin(), v.end(), [](int x) {
        return x % 2 == 0;
    });

    cout << even;
}
```

처음 배울 때 자주 하는 실수

- 여러 값의 빈도수를 반복해서 세면 비효율적
- 조건이 복잡하면 count_if 사용
- 정렬된 데이터의 특정 값 개수는 lower_bound 조합도 가능

비슷한 항목과 차이

- find는 존재 위치를 찾음
- map/unordered_map은 여러 값 빈도표에 적합
- count_if는 조건 기반 계산

한 줄 정리: count은(는) '특정 값의 등장 횟수 계산' 상황에서 먼저 떠올리면 좋은 STL입니다.